

Arbres de Merkle et vérification légère dans Bitcoin

Il est rappelé que le jury n'exige pas une compréhension exhaustive du texte. Vous êtes laissé(e) libre d'organiser votre présentation comme vous l'entendez. Il vous est conseillé de mettre en valeur vos connaissances et votre culture scientifique à partir du fil conducteur constitué par le texte. Les *suggestions de développement*, en fin de texte, ne sont là que pour vous aider ; vous ne devez pas vous sentir tenu(e) de les suivre, ni de les traiter toutes. Le jury appréciera que votre exposé s'accompagne d'**exemples, d'implémentations et d'expériences** menés sur ordinateur (Python conseillé : `hashlib`, `numpy`, `matplotlib`).

1 Le problème : vérifier sans tout télécharger

Une chaîne de blocs de type Bitcoin est répliquée sur des milliers de machines indépendantes ; sa taille se compte en centaines de gigaoctets. Un utilisateur muni d'un simple téléphone ne peut ni stocker ni revalider l'intégralité de l'historique. Pourtant, il souhaite s'assurer qu'une transaction précise — par exemple un paiement qui le concerne — a bien été incluse dans un bloc accepté par le réseau. Le problème central est donc celui d'une *vérification légère* : convaincre un client de faible capacité de l'*appartenance* d'une donnée à un très grand ensemble, à l'aide d'un *certificat* de taille minimale, sans lui transmettre l'ensemble entier ni exiger qu'il le recalcule.

La solution retenue par Bitcoin (S. Nakamoto, 2008), et devenue un standard, repose sur une structure de données *authentifiée* : l'*arbre de Merkle* (R. Merkle, 1979). Le fil conducteur de ce texte relie une brique cryptographique (la fonction de hachage), une structure arborescente et sa complexité, une preuve de correction fondée sur une *réduction*, et enfin un modèle probabiliste de la *preuve de travail* qui sécurise l'ajout des blocs. Chaque notion se prête à une implémentation et à une expérimentation numérique.

2 Hachage cryptographique et collisions

Bitcoin identifie chaque objet par son *double SHA-256*

$$H(m) = \text{SHA256}(\text{SHA256}(m)) \in \{0,1\}^{256},$$

que l'on peut voir comme une application $H : \{0,1\}^* \rightarrow \{0,1\}^{256}$. Une transaction t est résumée par son identifiant $\text{txid} = H(t)$. Deux propriétés opposées coexistent.

Proposition 1 (Existence des collisions). *L'application H n'est pas injective : il existe des messages distincts de même haché.*

Proof. L'ensemble de départ est infini, l'ensemble d'arrivée fini (de cardinal 2^{256}) ; par le principe des tiroirs, H ne peut être injective. □

Les collisions existent donc, mais on postule qu'elles sont *introuvables en pratique* : il n'existe pas d'algorithme efficace produisant $m \neq m'$ avec $H(m) = H(m')$. Cette *résistance aux collisions* est une hypothèse *calculatoire*, et non un théorème. Sa force se mesure par l'attaque générique la plus simple, que le modèle de l'*oracle aléatoire* permet d'analyser.

Proposition 2 (Borne des anniversaires). *En tirant k valeurs indépendantes et uniformes parmi $M = 2^b$, la probabilité d'absence de collision vaut $q_k = \prod_{i=0}^{k-1} (1 - i/M)$ et vérifie $q_k \leq \exp(-k(k-1)/(2M))$. Une collision devient probable dès que $k \gtrsim 1,177 \sqrt{M} = 1,177 \cdot 2^{b/2}$.*

Proof. Le i -ième tirage évite les i valeurs déjà obtenues avec probabilité $1 - i/M$; par indépendance on obtient le produit. L'inégalité $1 - x \leq e^{-x}$ donne $q_k \leq \exp(-\sum_{i=0}^{k-1} i/M) = \exp(-k(k-1)/(2M))$, quantité $\leq 1/2$ dès que $k(k-1) \geq 2M \ln 2$. $\square$

Ainsi une empreinte sur b bits n'offre qu'une résistance de l'ordre de $2^{b/2}$ opérations : le choix $b = 256$ vise une marge de 2^{128} , hors de portée. Le cas $M = 365$ n'est autre que le célèbre *paradoxe des anniversaires* (23 personnes suffisent). Ces seuils s'observent aisément par une expérience de Monte-Carlo sur une empreinte tronquée à b bits.

```
import hashlib, random
def H(m: bytes) -> bytes:
    return hashlib.sha256(hashlib.sha256(m).digest()).digest()
def tronque(h: bytes, b: int) -> int: # b premiers bits de h
    return int.from_bytes(h, "big") >> (256 - b)
# On peut mesurer le nombre de tirages avant la premiere collision, pour b=16,24,32,
# et le comparer a l'ordre de grandeur 2^{b/2} predit par la Proposition 2.
```

3 L'arbre de Merkle d'un bloc

Un bloc contient une liste ordonnée de transactions $t_0, \dots, t_{n-1}$. On les résume en un unique haché, la *racine de Merkle*, placé dans l'en-tête du bloc. La construction procède par niveaux : partant des feuilles $\text{txid}_i = H(t_i)$, on remplace chaque paire consécutive (a, b) par $H(a||b)$ (concaténation des octets) ; si un niveau contient un nombre impair de nœuds, on duplique son dernier élément avant l'appariement (règle propre à Bitcoin). On itère jusqu'à obtenir un seul haché.

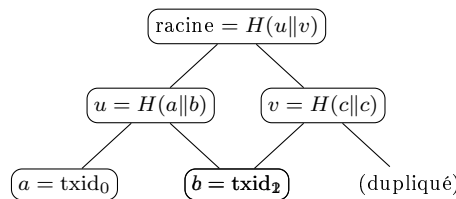


Figure 1 — arbre de Merkle d'un bloc à trois transactions : le dernier haché est dupliqué.

Proposition 3 (Coût de construction). *Le calcul de la racine de n transactions demande $O(n)$ évaluations de H , en temps et espace $O(n)$, et l'arbre a une hauteur $\lceil \log_2 n \rceil$.*

Proof. En notant $t_0 = n$ et $t_{k+1} = \lceil t_k/2 \rceil$ la taille du niveau $k+1$, on a $t_{k+1} \leq t_k/2 + 1$, d'où $t_k \leq n2^{-k} + 2$: la hauteur est $\lceil \log_2 n \rceil$ et il y a $O(\log n)$ niveaux. Le total des évaluations vaut $\sum_{k \geq 1} t_k \leq n + O(\log n) = O(n)$. $\square$

Remarque 1 (Une subtilité de sécurité). La règle de duplication a une conséquence inattendue : pour certains n , un bloc et un bloc obtenu en dupliquant réellement des transactions produisent la *même* racine. Cette ambiguïté (connue sous le nom de « CVE-2012-2459 ») illustre qu'une structure authentifiée doit être analysée avec soin : la racine seule ne garantit pas l'unicité de l'arbre sous-jacent.

4 Preuves d'inclusion et clients légers (SPV)

Un client *léger* (*Simplified Payment Verification*) ne conserve que les en-têtes des blocs, donc les racines. Pour lui prouver qu'une transaction t appartient à un bloc de racine R , on lui fournit une *preuve de Merkle* : la suite des hachés *frères* rencontrés en remontant de la feuille $H(t)$ jusqu'à la racine, chacun accompagné de son côté (gauche ou droite). Le client recombine ces hachés par H et accepte si et seulement s'il retrouve R .

La longueur d'une telle preuve est le nombre de niveaux, soit $\lceil \log_2 n \rceil = O(\log n)$: pour un bloc de dizaines de milliers de transactions, une poignée de hachés suffit, là où le téléchargement du bloc entier serait prohibitif. C'est précisément ce qui rend le modèle SPV viable sur un appareil modeste. La *sûreté* de ce mécanisme se ramène à la seule hypothèse sur H .

Théorème 1 (Sûreté de la vérification, ou *soundness*). *Supposons H résistante aux collisions. Si la vérification d'une preuve d'inclusion pour une valeur x aboutit à la racine authentique R , alors — sauf à exhiber une collision de H — la valeur x figure réellement parmi les transactions ayant produit R .*

Proof. Récurrence sur la hauteur H de l'arbre. Si $H = 0$, la racine *est* la feuille et la preuve est vide : l'acceptation impose $x = R$. Hérédité : la vérification calcule d'abord $r' = H(x\|f)$ (ou $H(f\|x)$) avec le premier frère f , puis poursuit : c'est une preuve d'inclusion valide de r' dans l'arbre de hauteur H des nœuds du niveau supérieur. Par hypothèse de récurrence, ou bien r' est un nœud authentique de ce niveau, ou bien on a produit une collision. Dans le premier cas, r' possède déjà un antécédent authentique par H issu de la construction ; l'égalité $H(x\|f) = r'$ fournit soit ce même couple (et alors x est la feuille attendue), soit un couple distinct de même image, c'est-à-dire une collision de H . En l'absence de collision calculable, x appartient donc bien au bloc. $\square$

Remarque 2. La sécurité de tout l'édifice se *réduit* ainsi à celle de H : forger une fausse preuve d'inclusion revient à calculer une collision (Proposition 2). Cette réduction est la véritable justification du dimensionnement $b = 256$.

5 La preuve de travail : un modèle probabiliste

Rester léger suppose de faire confiance à la chaîne d'en-têtes reçue. Bitcoin rend cette chaîne coûteuse à falsifier par la *preuve de travail*. L'en-tête d'un bloc contient un entier libre, le *nonce* ν ; *miner* le bloc, c'est trouver ν tel que $H(e_\nu)$, interprété comme un entier sur 256 bits, soit inférieur à une *cible* T . Dans le modèle de l'oracle aléatoire, chaque essai réussit indépendamment avec probabilité $p = T/2^{256}$.

Proposition 4 (Coût du minage). *Le nombre N d'essais jusqu'au premier succès suit une loi géométrique de paramètre p ; son espérance est $\mathbb{E}[N] = 1/p$, et le minage termine presque sûrement ($\mathbb{P}(N = \infty) = \lim_k (1 - p)^k = 0$).*

Le minage est donc un algorithme *probabiliste* de type Las Vegas : il rend toujours un résultat correct, mais son temps d'exécution est aléatoire, d'espérance $1/p$. En pratique, on fixe la difficulté en exigeant que le haché commence par k bits nuls, soit $p = 2^{-k}$ et $\mathbb{E}[N] = 2^k$; la distribution de N et cette espérance s'observent directement par simulation. Le réseau ajuste périodiquement T pour maintenir un intervalle moyen constant entre blocs malgré la variation de la puissance de calcul.

```
import itertools
def mine(entete: bytes, k: int) -> tuple[int, bytes]:
    for nu in itertools.count():
```

```

h = H(entete + nu.to_bytes(4, "big"))
if int.from_bytes(h, "big") >> (256 - k) == 0: # k bits de tete nuls
    return nu, h
# En repetant le minage, on peut estimer E[N] et tracer l'histogramme de N (loi
  geometrique).

```

Enfin, la *sécurité* de la chaîne se lit comme une course : un attaquant de puissance relative $\alpha < 1/2$, parti avec z blocs de retard, ne rattrape la chaîne honnête qu'avec une probabilité qui *décroît exponentiellement* en z . C'est ce qui fonde la règle des *confirmations* (attendre plusieurs blocs avant de tenir un paiement pour définitif), et ce qui relie la vérification légère du client SPV à la robustesse économique du consensus.

6 Prolongements et structures voisines

L'arbre de Merkle binaire de Bitcoin admet de nombreuses variantes. Ethereum emploie un arbre de *Patricia-Merkle* (mêlant arbre préfixe et hachage) pour indexer un état modifiable ; les *arbres de Verkle* remplacent le hachage par des *engagements polynomiaux* et abaissent la taille des preuves de $O(\log n)$ à $O(1)$. Plus généralement, la vérification « à faible coût » illustrée ici est l'idée directrice des *preuves à divulgation nulle* et des *rollups* : déléguer un calcul massif tout en n'en vérifiant qu'un certificat.

Suggestions pour le développement

Les pistes suivantes, largement indépendantes, ne sont qu'indicatives ; on n'en traitera qu'une partie, en privilégiant celles que l'on peut à la fois exposer, illustrer par une implémentation et éprouver par une simulation.

- Modéliser et implémenter la construction de la racine de Merkle d'un bloc, la génération et la vérification des preuves d'inclusion ; mettre en évidence expérimentalement la croissance logarithmique de la taille des certificats.
- Étudier la Proposition 2, en discuter l'équivalent asymptotique et la confronter à une expérience de Monte-Carlo sur une empreinte tronquée ; en déduire une interprétation du dimensionnement à 256 bits.
- Analyser en détail le Théorème 1 et expliciter la *réduction* : comment une fausse preuve fournit-elle, de manière effective, une collision ?
- Approfondir la Remarque 1 : caractériser les tailles de blocs sensibles à l'ambiguïté de la duplication et proposer une parade (vérification du nombre de feuilles).
- Étudier le modèle probabiliste du minage (Proposition 4) : distribution de N , effet de l'ajustement de la cible, temps moyen entre blocs pour une puissance donnée.
- Simuler la « course entre chaînes » et estimer la probabilité de rattrapage en fonction de α et z ; relier au choix du nombre de confirmations.
- Comparer, du point de vue de la complexité et de la taille des preuves, l'arbre binaire de Bitcoin, un arbre k -aire, l'arbre de Patricia-Merkle et les engagements de type Verkle.
- Situer la résistance aux collisions dans le cadre des *fonctions à sens unique* et de la cryptographie à sécurité prouvée.

Références

1. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
2. R. C. Merkle, *A digital signature based on a conventional encryption function*, CRYPTO '87.

3. A. M. Antonopoulos, *Mastering Bitcoin*, O'Reilly ; A. M. Antonopoulos, G. Wood, *Mastering Ethereum*, O'Reilly.
4. J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, CRC Press.